

Getting Started with Groq API - From Zero to Production-Grade Code

🔗 **Builds on:** `LLM_APIs_Basics.md` (API concepts, JSON response anatomy, API key security) 🎯 **Goal:** Go from having a Groq API key to writing clean, production-ready Python code that calls LLMs

Table of Contents

1. [Project Setup \(The Right Way with uv\)](#)
 2. [Secure API Key Management](#)
 3. [Your First Groq API Call](#)
 4. [Understanding the Response Object](#)
 5. [The Messages Array - Roles Explained](#)
 6. [Inference Parameters - Controlling the Model](#)
 7. [Available Models on Groq](#)
 8. [Building a Multi-Turn Conversation \(Chat\)](#)
 9. [Streaming Responses](#)
 10. [Error Handling & Retries](#)
 11. [Tracking Token Usage & Cost](#)
 12. [Putting It All Together - A Production-Grade Client](#)
 13. [Key Takeaways](#)
-

1. Project Setup (The Right Way with uv)

You're using `uv` as your package manager — here's the correct way to set up a new project.

```
bash
```

```
# Create a new project directory
mkdir groq-api-practice
cd groq-api-practice

# Initialize uv project (creates pyproject.toml)
uv init

# Add Groq SDK and dotenv for secret management
uv add groq python-dotenv

# Your project structure should look like:
# groq-api-practice/
# └─ pyproject.toml      ← project config + dependencies
# └─ .python-version    ← pins Python version
# └─ .env               ← your secrets (NEVER commit this)
# └─ .gitignore         ← must include .env
# └─ main.py            ← your code
```

Create your `.gitignore` immediately:

```
# .gitignore
.env
__pycache__/
*.pyc
.venv/
```

⚠ Non-negotiable: Create `.gitignore` BEFORE your first `git commit`. If `.env` gets committed even once, your API key is exposed in git history even after deletion.

2. Secure API Key Management

Step 1 — Store the key in `.env`

```
bash

# .env (never commit this file)
GROQ_API_KEY=gsk_your_actual_key_here
```

Step 2 — Load it in Python using `python-dotenv`

```
python
```

```
# At the top of every script that needs API access
import os
from dotenv import load_dotenv

load_dotenv() # Reads .env file and loads into environment

api_key = os.environ.get("GROQ_API_KEY")

# Always validate – fail early with a clear error
if not api_key:
    raise ValueError("GROQ_API_KEY not found. Check your .env file.")
```

Why this pattern matters in production

Development → .env file (on your machine only)
Staging → Environment variables set in the server config
Production → AWS Secrets Manager / Azure Key Vault / HashiCorp Vault

Your code reads from `os.environ` in ALL environments.
The SOURCE of the secret changes, but the code never does.

3. Your First Groq API Call

python

```

import os
from dotenv import load_dotenv
from groq import Groq

# Load environment variables
load_dotenv()

# Initialize the client - reads GROQ_API_KEY from environment automatically
client = Groq()

# Make your first API call
response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[
        {
            "role": "system",
            "content": "You are a helpful assistant. Be concise and clear."
        },
        {
            "role": "user",
            "content": "What is a large language model?"
        }
    ]
)

# Extract and print the response text
print(response.choices[0].message.content)

```

Run it:

```

bash

uv run main.py

```

Expected output:

A large language model (LLM) is a type of AI model trained on vast amounts of text data to understand and generate human language. Examples include GPT-4, Claude, and Llama. They can perform tasks like answering questions, writing, summarizing, and coding through natural language prompts.

 **Notice:** `Groq()` with no arguments automatically looks for `GROQ_API_KEY` in your environment. You don't pass the key directly — that's the correct pattern.

4. Understanding the Response Object

When you call the API, you get back a structured Python object (not raw JSON). Here's every field you need to know:

```
python

response = client.chat.completions.create(...)

# — The actual text response —————
response.choices[0].message.content      # The model's reply text
response.choices[0].message.role         # Always "assistant"

# — Why did the model stop? —————
response.choices[0].finish_reason
# "stop"    → Model finished naturally (most common)
# "length"  → Hit max_tokens limit (response was cut off)
# "error"   → Something went wrong

# — Token usage (important for cost tracking) —————
response.usage.prompt_tokens             # Tokens in your input
response.usage.completion_tokens         # Tokens in the response
response.usage.total_tokens              # Sum of both

# — Metadata —————
response.id                             # Unique ID for this call
response.model                           # Which model was actually used
response.created                         # Unix timestamp of the call

# — Full example —————
print(f"Response:      {response.choices[0].message.content}")
print(f"Stop reason:  {response.choices[0].finish_reason}")
print(f"Tokens used:   {response.usage.total_tokens}")
print(f"Model used:    {response.model}")
```

The `finish_reason` Check — Why It Matters

```
python
```

```

response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[...],
    max_tokens=50    # Very low limit to demonstrate truncation
)

if response.choices[0].finish_reason == "length":
    print("⚠️ WARNING: Response was truncated! Increase max_tokens.")
    # In production: log this, alert, or retry with higher limit
elif response.choices[0].finish_reason == "stop":
    print("✅ Response completed naturally.")

```

Enterprise Example 🏢:

A document summarization pipeline processes legal contracts. If `finish_reason == "length"`, the summary was cut off mid-sentence — an incomplete summary going into the system is worse than no summary. Always check this.

5. The Messages Array - Roles Explained

The `messages` array is the **entire conversation**. Every call you make includes the full history.

The Three Roles

python

```

messages = [
    # ROLE 1: system
    # -----
    # Sets the model's persona, behavior, and constraints.
    # Always comes first. Only one system message per conversation.
    {
        "role": "system",
        "content": """You are a senior software engineer at a fintech company.
        You answer questions about Python and system design.
        Always include code examples.
        Never discuss topics unrelated to software engineering."""
    },

    # ROLE 2: user
    # -----
    # The human's message. Can appear multiple times in history.
    {
        "role": "user",
        "content": "How should I handle database connection pooling?"
    },

    # ROLE 3: assistant
    # -----
    # The model's previous response. Added to history for multi-turn chats.
    # You create this from response.choices[0].message.content
    {
        "role": "assistant",
        "content": "For connection pooling in Python, I recommend using SQLAlchemy.."
    },

    # Next user message continues the conversation
    {
        "role": "user",
        "content": "Can you show me a code example with PostgreSQL?"
    }
]

```

Visual Flow of a Conversation

API Call 1:

[system] + [user: "Hello"] → model → [assistant: "Hi! How can I help?"]

API Call 2:

```
[system] + [user: "Hello"] + [assistant: "Hi!"] + [user: "Explain RAG"] → model  
→ [assistant: "RAG stands for..."]
```

API Call 3:

```
[system] + [user: "Hello"] + [assistant: "Hi!"] + [user: "Explain RAG"] +  
[assistant: "RAG stands for..."] + [user: "Give an example"] → model → [assistant:  
"Sure, here's an example..."]
```

Each API call receives the **full conversation history**. The model has no memory between calls — you are responsible for maintaining and passing the history.

6. Inference Parameters – Controlling the Model

These parameters go inside `client.chat.completions.create()`:

python

```
response = client.chat.completions.create(  
    model="llama3-8b-8192",  
    messages=[...],  
  
    temperature=0.7,      # 0.0 to 2.0 – controls randomness  
    max_tokens=1024,      # max tokens in the response  
    top_p=0.9,            # nucleus sampling (alternative to temperature)  
    stop=["###", "END"],  # stop generation when these strings appear  
    seed=42,              # for reproducible outputs (if supported)  
)
```

Temperature — The Most Important Parameter

temperature = 0.0

- └ Output is nearly deterministic
- └ Same prompt → same response every time
- └ Use for: data extraction, classification, code generation

temperature = 0.3 - 0.5

- └ Slight variation, mostly consistent
- └ Use for: customer support, Q&A, factual tasks

temperature = 0.7 - 0.9

- └ Creative, varied responses

└─ Use for: content writing, brainstorming, chat

temperature = 1.0 - 2.0

└─ Highly creative, sometimes unpredictable

└─ Use for: creative writing, poetry, idea generation

Practical Temperature Examples

python

For extracting structured data – use 0.0

```
response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[
        {"role": "system", "content": "Extract data as JSON. Return only JSON, no ex"},
        {"role": "user", "content": "Invoice from Acme Corp, $5,400, dated March 15"},
    ],
    temperature=0.0,    # ← Deterministic – same input = same JSON every time
    max_tokens=200
)
```

For creative product descriptions – use 0.8

```
response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[
        {"role": "system", "content": "Write engaging product descriptions."},
        {"role": "user", "content": "Write a description for a wireless ergonomic mo"},
    ],
    temperature=0.8,    # ← Creative – different variations on each run
    max_tokens=150
)
```

max_tokens — Always Set This

python

```
# ❌ Without max_tokens – model can generate very long responses
# This increases cost and latency unpredictably
response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[{"role": "user", "content": "Tell me everything about AI"}]
)

# ✅ Always set max_tokens – control cost and response length
response = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[{"role": "user", "content": "Tell me everything about AI"}],
    max_tokens=500 # Cap at 500 tokens (~375 words)
)
```

7. Available Models on Groq

```
python

# List available models programmatically
models = client.models.list()
for model in models.data:
    print(model.id)
```

Key Models and When to Use Them

Model	Speed	Context	Best For
llama3-8b-8192	Very Fast	8K tokens	Quick tasks, high volume
llama3-70b-8192	Fast	8K tokens	Better reasoning, complex tasks
llama-3.1-8b-instant	Fastest	128K tokens	Real-time apps, large context
llama-3.1-70b-versatile	Fast	128K tokens	Best quality + large context
mixtral-8x7b-32768	Fast	32K tokens	Long documents
gemma2-9b-it	Fast	8K tokens	Google's model, instruction-tuned

Model Selection Strategy

Need fastest response?	→ llama3-8b-8192
Need best quality?	→ llama-3.1-70b-versatile
Need to process long documents?	→ llama-3.1-8b-instant (128K context)
Need to balance both?	→ llama3-70b-8192

Make Model Configurable (Production Best Practice)

```
python

import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()

# ❌ Hardcoded model – hard to change later
model = "llama3-8b-8192"

# ✅ Configurable via environment – change without touching code
model = os.environ.get("GROQ_MODEL", "llama3-8b-8192") # default fallback

client = Groq()
response = client.chat.completions.create(
    model=model,
    messages=[...]
)
```

Now you can switch models via `.env` with zero code changes:

```
bash

# .env
GROQ_API_KEY=gsk_...
GROQ_MODEL=llama-3.1-70b-versatile
```

8. Building a Multi-Turn Conversation (Chat)

This is how you build an actual chatbot — maintaining conversation history across turns.

```
python
```

```

import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq()

def chat():
    """Simple interactive chatbot with conversation memory."""

    # Initialize conversation with system prompt
    conversation_history = [
        {
            "role": "system",
            "content": """You are a helpful AI assistant specializing in
Python programming. You give clear, practical answers with
code examples when relevant."""
        }
    ]

    print("🤖 Python Assistant Ready! Type 'quit' to exit.\n")

    while True:
        # Get user input
        user_input = input("You: ").strip()

        if user_input.lower() in ["quit", "exit", "bye"]:
            print("Assistant: Goodbye! Happy coding!")
            break

        if not user_input:
            continue

        # Add user message to history
        conversation_history.append({
            "role": "user",
            "content": user_input
        })

        # Call the API with full history
        response = client.chat.completions.create(
            model=os.environ.get("GROQ_MODEL", "llama3-8b-8192"),
            messages=conversation_history,

```

```
        temperature=0.5,
        max_tokens=1024
    )

    # Extract assistant response
    assistant_message = response.choices[0].message.content

    # Add assistant response to history (critical for memory)
    conversation_history.append({
        "role": "assistant",
        "content": assistant_message
    })

    print(f"\nAssistant: {assistant_message}\n")
    print(f"[Tokens used this turn: {response.usage.total_tokens}]")
    print(f"[Total messages in history: {len(conversation_history)}]\n")

if __name__ == "__main__":
    chat()
```

The Context Window Problem

Conversation history grows with every turn. Eventually it exceeds the model's context window.

```
Turn 1: 500 tokens
Turn 5: 2,000 tokens
Turn 20: 8,000 tokens ← hitting the 8K limit of llama3-8b-8192
Turn 21: ERROR – context window exceeded!
```

Production solution — sliding window:

```
python
```

```
def trim_history(history, max_messages=20):
    """
    Keep system prompt + last N messages.
    Prevents context window overflow in long conversations.
    """
    system_prompt = history[0]          # Always keep system prompt
    recent_messages = history[1:]       # Everything after system prompt

    if len(recent_messages) > max_messages:
        # Keep only the most recent messages
        recent_messages = recent_messages[-max_messages:]
        print(f"[History trimmed to last {max_messages} messages]")

    return [system_prompt] + recent_messages

# Usage in the chat loop:
conversation_history = trim_history(conversation_history, max_messages=20)
response = client.chat.completions.create(
    messages=conversation_history,
    ...
)
```

9. Streaming Responses

By default, the API waits until the **entire response is ready** before returning it. With streaming, tokens are delivered **as they are generated** — like watching someone type.

Why streaming matters

Without streaming:

User sends message → [waiting 3-5 seconds] → Full response appears at once
Feels slow, especially for long responses

With streaming:

User sends message → tokens appear immediately, word by word
Feels fast and responsive, like ChatGPT

python

```

import os
from dotenv import load_dotenv
from groq import Groq

load_dotenv()
client = Groq()

# stream=True enables streaming
stream = client.chat.completions.create(
    model="llama3-8b-8192",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain how the internet works in 3 paragraphs."},
    ],
    temperature=0.5,
    max_tokens=500,
    stream=True      # ← Enable streaming
)

# Collect full response for storage
full_response = ""

print("Assistant: ", end="", flush=True)

for chunk in stream:
    # Each chunk contains a small piece of the response
    delta = chunk.choices[0].delta.content
    if delta:
        print(delta, end="", flush=True)    # Print immediately without newline
        full_response += delta

print() # Final newline after streaming completes

# Now full_response contains the complete text for storage/processing
print(f"\n[Total characters: {len(full_response)}]")

```

Enterprise Example 🏢:

A customer support chat interface uses streaming so the response starts appearing within 200ms. Without streaming, customers wait 4-5 seconds for the full response — feels broken. With streaming, the UX matches modern chat expectations.

10. Error Handling & Retries

Production code must handle failures gracefully. Groq API calls can fail for several reasons:

```
python
```



```

import os
import time
from dotenv import load_dotenv
from groq import Groq
from groq import RateLimitError, APIConnectionError, APIStatusError

load_dotenv()
client = Groq()

def call_with_retry(messages, max_retries=3, model="llama3-8b-8192"):
    """
    Calls the Groq API with automatic retry on rate limit errors.
    Uses exponential backoff – waits longer after each failure.
    """
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model=model,
                messages=messages,
                temperature=0.5,
                max_tokens=1024
            )
            return response # Success – return immediately

        except RateLimitError as e:
            # HTTP 429 – you've hit the rate limit
            wait_time = (2 ** attempt) * 1 # 1s, 2s, 4s
            print(f"⚠️ Rate limit hit. Waiting {wait_time}s before retry {attempt+1}")
            time.sleep(wait_time)

        except APIConnectionError as e:
            # Network issue – internet down, DNS failure, etc.
            wait_time = (2 ** attempt) * 2 # 2s, 4s, 8s
            print(f"🔌 Connection error. Retrying in {wait_time}s... ({e})")
            time.sleep(wait_time)

        except APIStatusError as e:
            # HTTP 4xx/5xx errors
            if e.status_code == 400:
                # Bad request – your prompt is malformed, don't retry
                raise ValueError(f"Bad request to API: {e.message}") from e
            elif e.status_code == 401:
                # Unauthorized – invalid API key, don't retry

```

```

        raise PermissionError("Invalid API key. Check GROQ_API_KEY.") from e
    elif e.status_code >= 500:
        # Server error – retry
        wait_time = (2 ** attempt) * 2
        print(f"🔴 Server error {e.status_code}. Retrying in {wait_time}s...")
        time.sleep(wait_time)
    else:
        raise # Unknown error – don't retry

# All retries exhausted
raise RuntimeError(f"API call failed after {max_retries} attempts.")

# Usage
try:
    response = call_with_retry(
        messages=[
            {"role": "system", "content": "You are a helpful assistant."},
            {"role": "user", "content": "What is RAG in GenAI?"}
        ]
    )
    print(response.choices[0].message.content)

except RuntimeError as e:
    print(f"❌ Failed: {e}")
    # In production: log this, alert the team, return a fallback response

```

Error Types at a Glance

Error	HTTP Code	Cause	Should Retry?
<code>RateLimitError</code>	429	Too many requests	✅ Yes, with backoff
<code>APIConnectionError</code>	—	Network issue	✅ Yes
<code>APIStatusError 400</code>	400	Bad prompt/request	❌ No — fix the prompt
<code>APIStatusError 401</code>	401	Invalid API key	❌ No — fix the key
<code>APIStatusError 500</code>	500	Groq server error	✅ Yes

11. Tracking Token Usage & Cost

Tokens = money. Always track them.

```
python
```

```

import os
from dotenv import load_dotenv
from groq import Groq
from datetime import datetime

load_dotenv()
client = Groq()

# Simple token usage tracker
class UsageTracker:
    def __init__(self):
        self.total_prompt_tokens = 0
        self.total_completion_tokens = 0
        self.total_calls = 0
        self.session_start = datetime.now()

    def record(self, usage):
        self.total_prompt_tokens += usage.prompt_tokens
        self.total_completion_tokens += usage.completion_tokens
        self.total_calls += 1

    def report(self):
        total = self.total_prompt_tokens + self.total_completion_tokens
        duration = (datetime.now() - self.session_start).seconds
        print(f"\n{'='*40}")
        print(f"📊 SESSION USAGE REPORT")
        print(f"{'='*40}")
        print(f"Total API calls:           {self.total_calls}")
        print(f"Prompt tokens:             {self.total_prompt_tokens:,}")
        print(f"Completion tokens:         {self.total_completion_tokens:,}")
        print(f"Total tokens:              {total:,}")
        print(f"Session duration:          {duration}s")
        # Groq's Llama3-8b is approximately $0.05 per million tokens
        estimated_cost = total * 0.00000005
        print(f"Estimated cost:             ${estimated_cost:.6f}")
        print(f"{'='*40}")

tracker = UsageTracker()

# Make several calls
questions = [
    "What is a vector database?",

```

```

    "What is RAG?",
    "What is an AI agent?"
]

for question in questions:
    response = client.chat.completions.create(
        model="llama3-8b-8192",
        messages=[
            {"role": "system", "content": "Answer in 2 sentences maximum."},
            {"role": "user", "content": question}
        ],
        max_tokens=100
    )
    tracker.record(response.usage)
    print(f"Q: {question}")
    print(f"A: {response.choices[0].message.content}\n")

tracker.report()

```

Output:

```

Q: What is a vector database?
A: A vector database stores data as high-dimensional vectors...

Q: What is RAG?
A: RAG (Retrieval-Augmented Generation) combines information retrieval...

Q: What is an AI agent?
A: An AI agent is a system that uses an LLM to autonomously...

```

```

=====
📊 SESSION USAGE REPORT
=====
Total API calls:      3
Prompt tokens:       87
Completion tokens:   156
Total tokens:        243
Session duration:    4s
Estimated cost:      $0.000012
=====

```

12. Putting It All Together - A Production-Grade Client

Here's a complete, reusable Groq client that incorporates everything above:

```
python
```

```

# groq_client.py – Reusable production-grade Groq client

import os
import time
import logging
from datetime import datetime
from dotenv import load_dotenv
from groq import Groq, RateLimitError, APIConnectionError, APIStatusError

load_dotenv()

# Set up logging – in production this goes to a log file or monitoring system
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s | %(levelname)s | %(message)s'
)
logger = logging.getLogger(__name__)

class GroqClient:
    """
    Production-grade wrapper around the Groq API.
    Handles: auth, retries, token tracking, logging, streaming.
    """

    def __init__(self):
        api_key = os.environ.get("GROQ_API_KEY")
        if not api_key:
            raise ValueError("GROQ_API_KEY not set in environment.")

        self.client = Groq(api_key=api_key)
        self.default_model = os.environ.get("GROQ_MODEL", "llama3-8b-8192")
        self.total_tokens_used = 0
        self.total_calls = 0

        logger.info(f"GroqClient initialized | model={self.default_model}")

    def complete(
        self,
        messages: list,
        model: str = None,
        temperature: float = 0.5,
        max_tokens: int = 1024,

```

```

        stream: bool = False,
        max_retries: int = 3
    ) -> str:
        """
        Send messages to Groq and return the response text.

        Args:
            messages: List of {role, content} dicts
            model: Model ID (defaults to GROQ_MODEL env var)
            temperature: 0.0-2.0, controls randomness
            max_tokens: Maximum response length
            stream: Whether to stream the response
            max_retries: Number of retry attempts on failure

        Returns:
            Response text as a string
        """
        model = model or self.default_model

        for attempt in range(max_retries):
            try:
                if stream:
                    return self._stream_complete(messages, model, temperature, max_t

                response = self.client.chat.completions.create(
                    model=model,
                    messages=messages,
                    temperature=temperature,
                    max_tokens=max_tokens
                )

                # Track usage
                self.total_tokens_used += response.usage.total_tokens
                self.total_calls += 1

                # Warn if response was truncated
                if response.choices[0].finish_reason == "length":
                    logger.warning(f"Response truncated at {max_tokens} tokens. Cons

            logger.info(
                f"API call successful | "
                f"model={model} | "
                f"tokens={response.usage.total_tokens} | "
                f"total_calls={self.total_calls}"

```



```

    )

    return response.choices[0].message.content

except RateLimitError:
    wait = 2 ** attempt
    logger.warning(f"Rate limit hit. Retry {attempt+1}/{max_retries} in
    time.sleep(wait)

except APIConnectionError as e:
    wait = 2 ** attempt * 2
    logger.warning(f"Connection error. Retry {attempt+1}/{max_retries} i
    time.sleep(wait)

except APIStatusError as e:
    if e.status_code in (400, 401):
        logger.error(f"Non-retryable error {e.status_code}: {e.message}")
        raise
    wait = 2 ** attempt * 2
    logger.warning(f"Server error {e.status_code}. Retry {attempt+1}/{ma
    time.sleep(wait)

raise RuntimeError(f"Groq API failed after {max_retries} retries.")

def _stream_complete(self, messages, model, temperature, max_tokens) -> str:
    """Internal method to handle streaming responses."""
    stream = self.client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        max_tokens=max_tokens,
        stream=True
    )
    full_response = ""
    for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            print(delta, end="", flush=True)
            full_response += delta
    print()
    return full_response

def usage_report(self):
    """Print a summary of tokens used this session."""

```

```
logger.info(  
    f"Session summary | "  
    f"total_calls={self.total_calls} | "  
    f"total_tokens={self.total_tokens_used:},"  
)
```

— Example usage —

```
if __name__ == "__main__":  
    groq = GroqClient()
```

1. Simple Q&A

```
answer = groq.complete(  
    messages=[  
        {"role": "system", "content": "You are a helpful assistant. Be concise."  
        {"role": "user", "content": "What is a transformer in AI?"}  
    ],  
    temperature=0.3,  
    max_tokens=200  
)  
print(f"Answer: {answer}\n")
```

2. Structured data extraction

```
invoice_text = "Invoice from TechCorp, amount $12,500, due April 30, 2025, for c  
extracted = groq.complete(  
    messages=[  
        {  
            "role": "system",  
            "content": "Extract invoice data as JSON. Return only valid JSON wit  
        },  
        {"role": "user", "content": invoice_text}  
    ],  
    temperature=0.0, # Deterministic for structured extraction  
    max_tokens=150  
)  
print(f"Extracted: {extracted}\n")
```

3. Streaming response

```
print("Streaming response:")  
groq.complete(  
    messages=[  
        {"role": "system", "content": "You are a helpful assistant."},  
        {"role": "user", "content": "List 3 benefits of using uv for Python proj
```

```
    ],  
    stream=True,  
    max_tokens=300  
)  
  
# 4. Usage report  
groq.usage_report()
```

13. Key Takeaways

- ✓ Always use uv to manage dependencies
- ✓ Always store API keys in .env – never in code
- ✓ Always add .env to .gitignore before first commit
- ✓ Use Groq() with no args – it reads GROQ_API_KEY automatically
- ✓ Always check finish_reason – "length" means response was cut off
- ✓ Always set max_tokens – prevents runaway costs
- ✓ Maintain conversation history manually for multi-turn chat
- ✓ Trim history to avoid context window overflow
- ✓ Use streaming for user-facing interfaces
- ✓ Implement retry with exponential backoff for rate limits
- ✓ Track token usage on every call – tokens = cost
- ✓ Make model name configurable via env var, not hardcoded

What Comes Next

Groq API Basics (this file) ✓

↓

Prompt Engineering → System prompts, few-shot, chain-of-thought

↓

Structured Outputs → JSON mode, parsing, validation with Pydantic

↓

RAG Systems → Embeddings, vector DBs, retrieval pipelines

↓

Agentic AI → Tool use, LangGraph, multi-agent systems